SISTEMAS OPERACIONAIS

SINCRONIZAÇÃO DE PROCESSOS

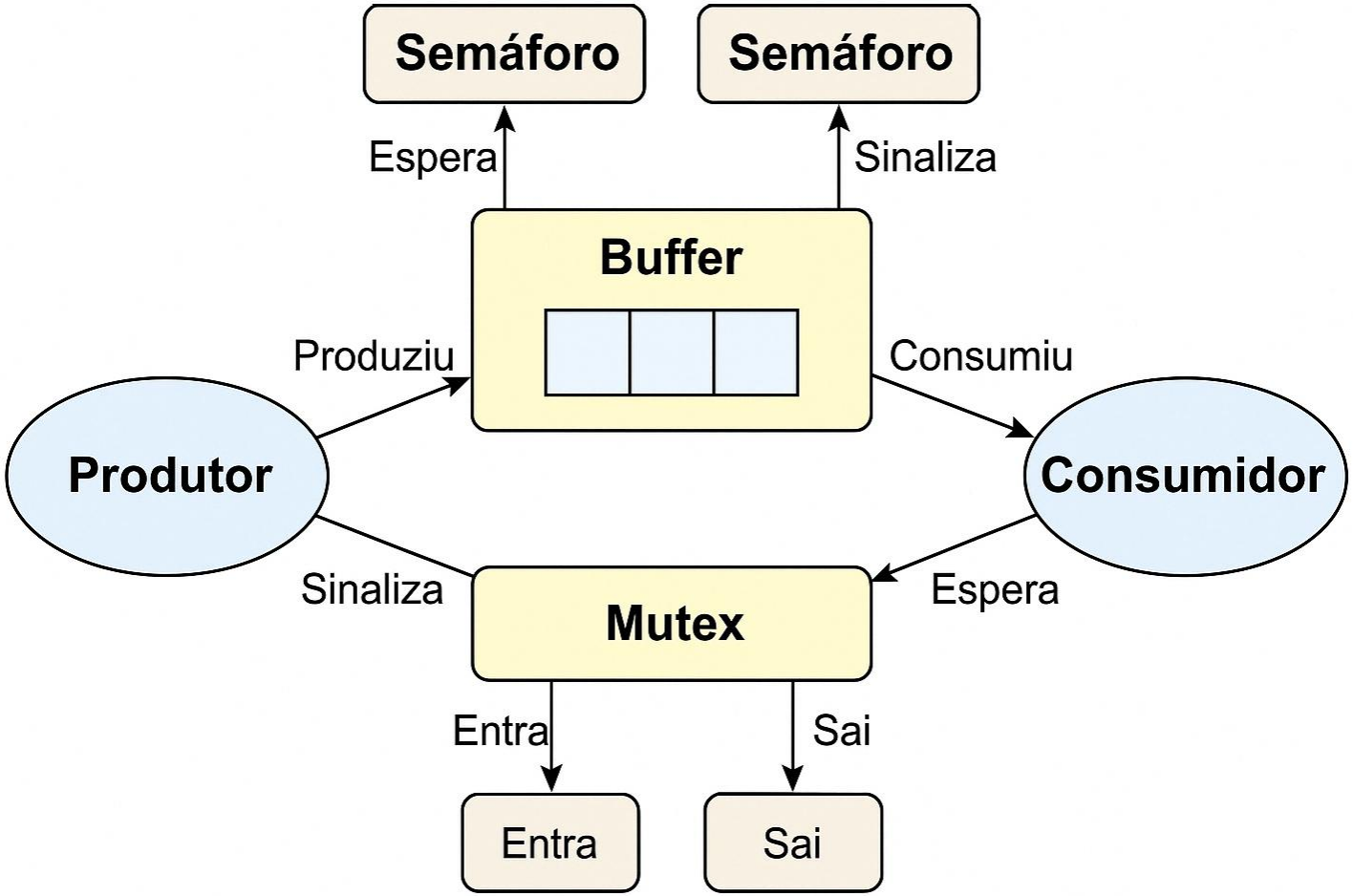
Análise do problema da Montanha-Russa

Andrei Gustavo de Lima Macedo e Caroline Balko Piovesam

O que é sincronização entre processos? Em que contexto é relevante para Sistemas Operacionais?

É a coordenação de fluxos concorrentes (threads ou processos) para garantir que acessem dados compartilhados de forma correta e segura. É relevante porque o sistema operacional alterna a CPU entre as threads de forma imprevisível (interleaving), e sem controle, acessos simultâneos corrompem o estado da memória.

O que é um mutex e o que é um semáforo?



- **Mutex:** Funciona como uma chave única de exclusão mútua. Apenas a thread que segura o mutex pode entrar na seção crítica; as outras ficam bloqueadas esperando.

- **Semáforo:** É um contador de permissões numéricas (0, 1, 2...). Ele serve para controlar quantidades limitadas de um recurso ou sinalizar a conclusão de tarefas entre threads diferentes.

Qual a relação do problema escolhido por vocês com a dinâmica real de um SO?

O problema da Montanha-Russa simula o processamento em lotes (*batch*) e gerenciamento de buffers. O carrinho atua como um recurso pesado que só é acionado quando um lote completo de requisições (passageiros) se acumula, otimizando o uso do hardware e bloqueando novas entradas durante o processamento.

O que pode acontecer se você não estiver atento a sincronização de processos?

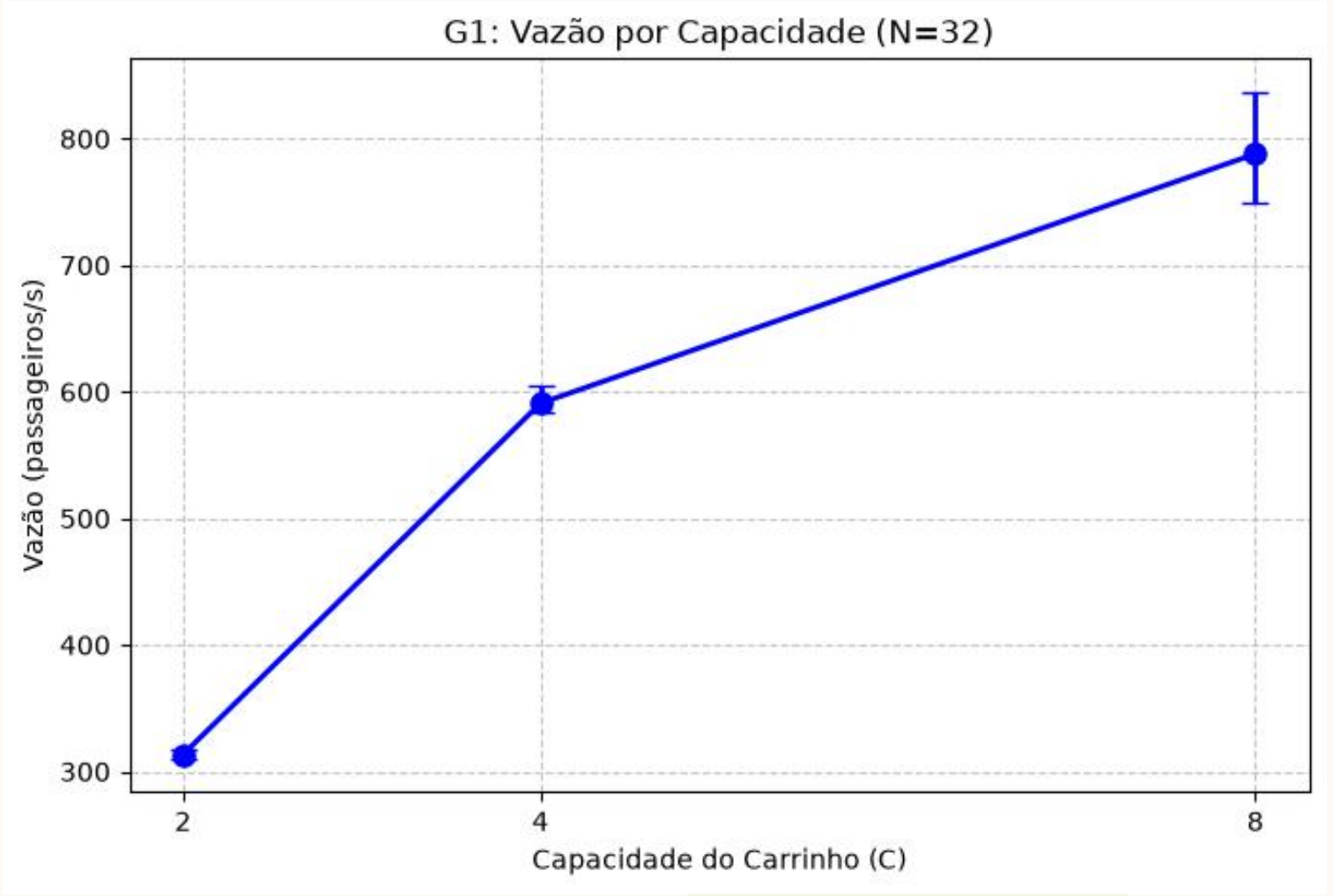
Ocorrem falhas graves de concorrência, como condições de corrida (*race conditions*), contadores corrompidos, violação de regras de capacidade, além de travamentos totais do sistema (*deadlocks*) ou exclusão indefinida de threads (*starvation*).

Existem alternativas à solução que escolheram? A solução escolhida segue os bons princípios para problemas de sincronização? Quais são esses princípios?

Sim, poderíamos usar Semáforos Contadores em vez de Variáveis de Condição. A nossa solução segue os bons princípios porque cumpre os três pilares fundamentais:

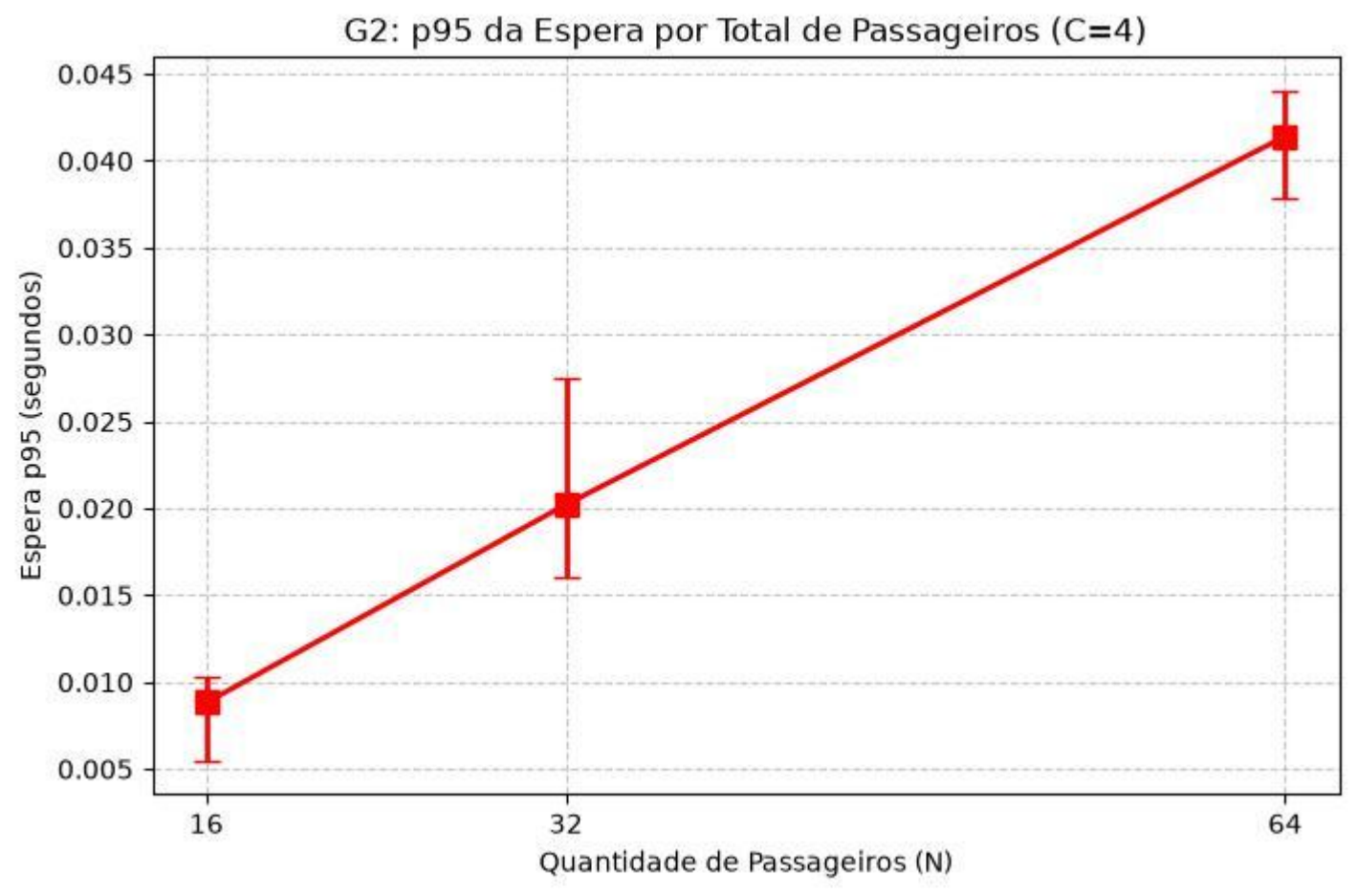
1. **Exclusão Mútua:** Protege as variáveis globais com Mutex.
2. **Progresso:** Utiliza espera bloqueante (pthread_cond_wait) em vez de laços vazios, liberando a CPU.
3. **Espera Limitada:** Garante que o p95 da espera seja baixo e estável, evitando *starvation*.

Gráfico 1 (G1 - Vazão por Capacidade)



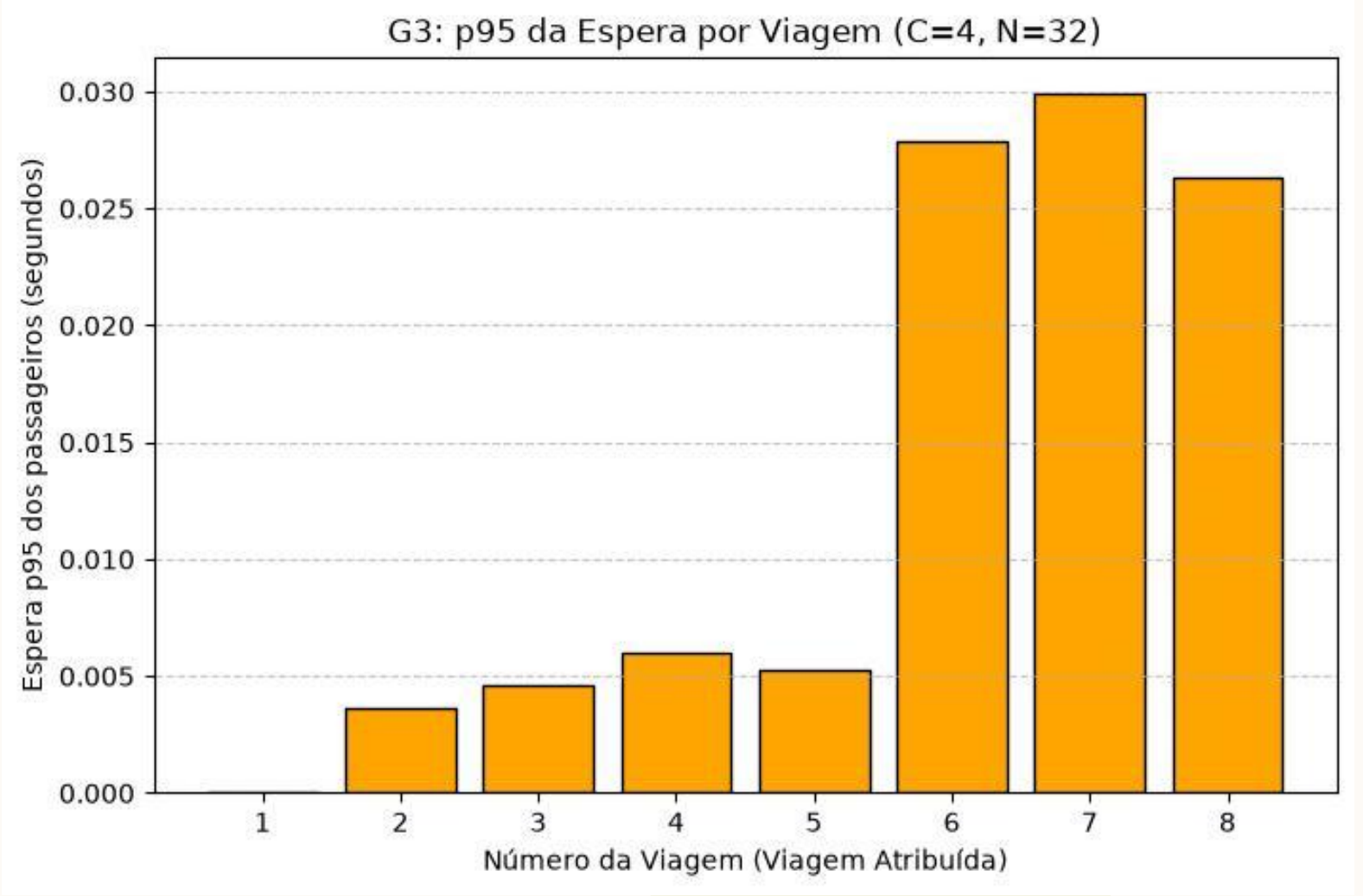
Mostra que carrinhos maiores aumentam a vazão de passageiros por segundo, pois o custo fixo de tempo do passeio é diluído em mais pessoas transportadas por viagem.

Gráfico 2 (G2 - p95 da Espera por Total de Passageiros)



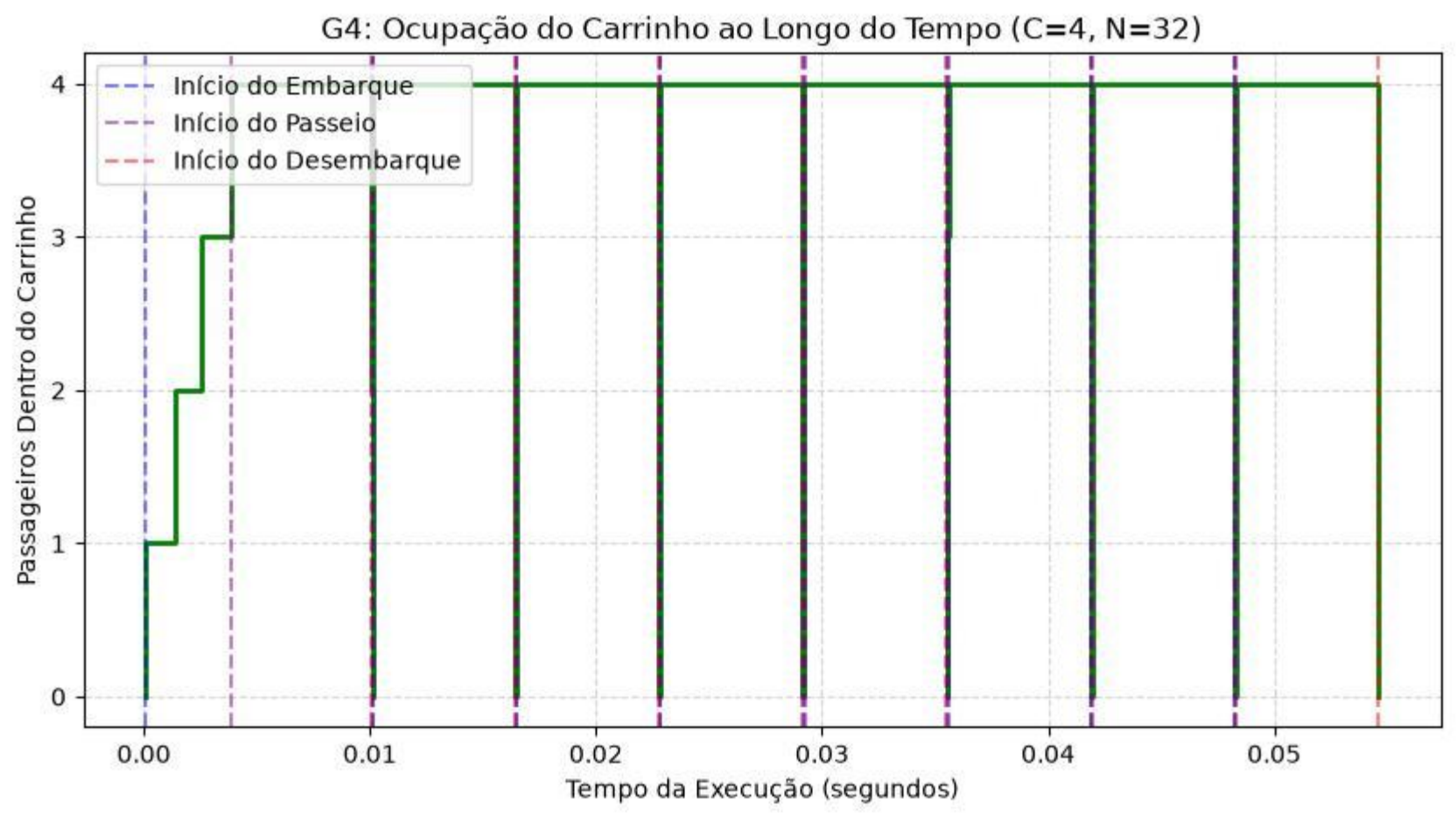
Revela que o tempo de espera do percentil 95 cresce de forma linear conforme o número total de passageiros aumenta, devido ao maior volume acumulado na fila.

Gráfico 3 (G3 - p95 da Espera por Viagem)



Demonstra que as últimas viagens sofrem mais atrasos e esperas maiores, refletindo o acúmulo gradual de passageiros na fila, já que a taxa de chegada supera o ritmo das viagens.

Gráfico 4 (G4 - Ocupação do Carrinho ao Longo do Tempo)



Comprova visualmente que as regras do problema foram respeitadas: o carrinho só parte cheio (em 4 lugares), fica totalmente isolado durante o passeio e esvazia por completo antes do próximo embarque.